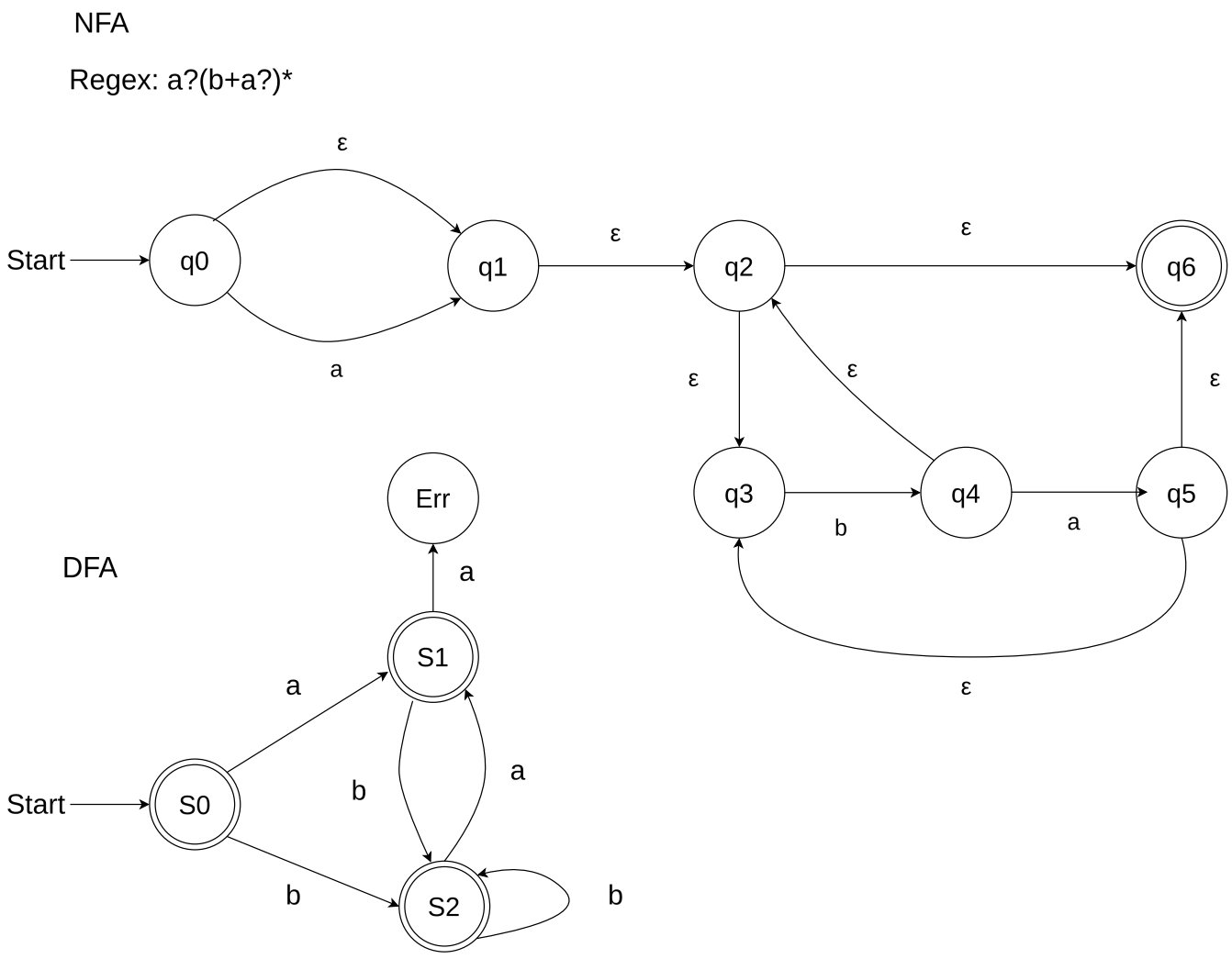
Exercise 3.2

Write a regular expression that recognizes all sequences consisting of 'a' and 'b' where two 'a's are always separated by at least one 'b'. For instance, these four strings are legal: 'b', 'ba', 'ababbbaba'; but these two strings are illegal: 'aa', 'babaa'

Construct the corresponding NFA. Try to find a DFA corresponding ot the NFA.

Regex: a?(b+a?)*

See `diagrams/ex3_2.drawio.svg` for NFA & DFA.



DFA state	move(a)	move(b)	NDFA states
S0	S1	S2	{ 1, 2, 3, 4, 6 }
S1	Err	S2	{ 2, 3, 4, 6 }
S2	S1	S2	{ 2, 3, 4, 6 }

Figure 1: NFA and DFA

Exercise 3.3

Write out the rightmost derivation of the string below from the expression grammar at the end of Sect. 3.6.5, corresponding to ExprPar.fsy. Take note of the sequence of grammar rules (A-I) used.

"let z = (17) in z + 2 * 3 end EOF"

`Note: Added parenthesis, square brackets, and values for the sake of readability`

```
Main
=> Expr EOF                                                                                    (rule A)
=> LET NAME(z) EQ Expr IN Expr END EOF                                                         (rule F)
=> LET NAME(z) EQ Expr IN (Expr PLUS Expr) END EOF                                             (rule H)
=> LET NAME(z) EQ Expr IN [Expr PLUS (Expr TIMES Expr)] END EOF                                (rule G)
=> LET NAME(z) EQ Expr IN (Expr PLUS [Expr TIMES CSTINT(3)]) END EOF                           (rule C)
=> LET NAME(z) EQ Expr IN (Expr PLUS [CSTINT(2) TIMES CSTINT(3)]) END EOF                      (rule C)
=> LET NAME(z) EQ Expr IN (NAME(z) PLUS [CSTINT(2) TIMES CSTINT(3)]) END EOF                   (rule B)
=> LET NAME(z) EQ (LPAR Expr RPAR) IN (NAME(z) PLUS [CSTINT(2) TIMES CSTINT(3)]) END EOF       (rule E)
=> LET NAME(z) EQ (LPAR CSTINT(17) RPAR) IN (NAME(z) PLUS [CSTINT(2) TIMES CSTINT(3)]) END EOF (rule C)
```

Exercise 3.4 Draw the above derivation as a tree

See diagram `diagrams/derivationtree.drawio.png`

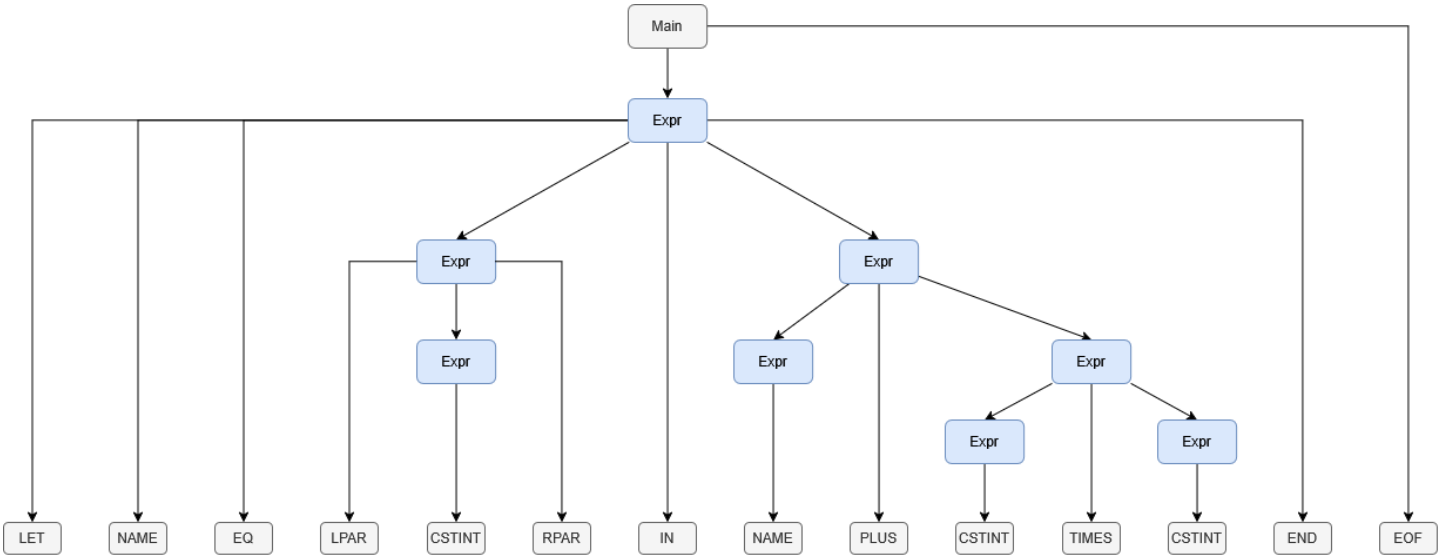


Figure 2: Derivation tree